AWS

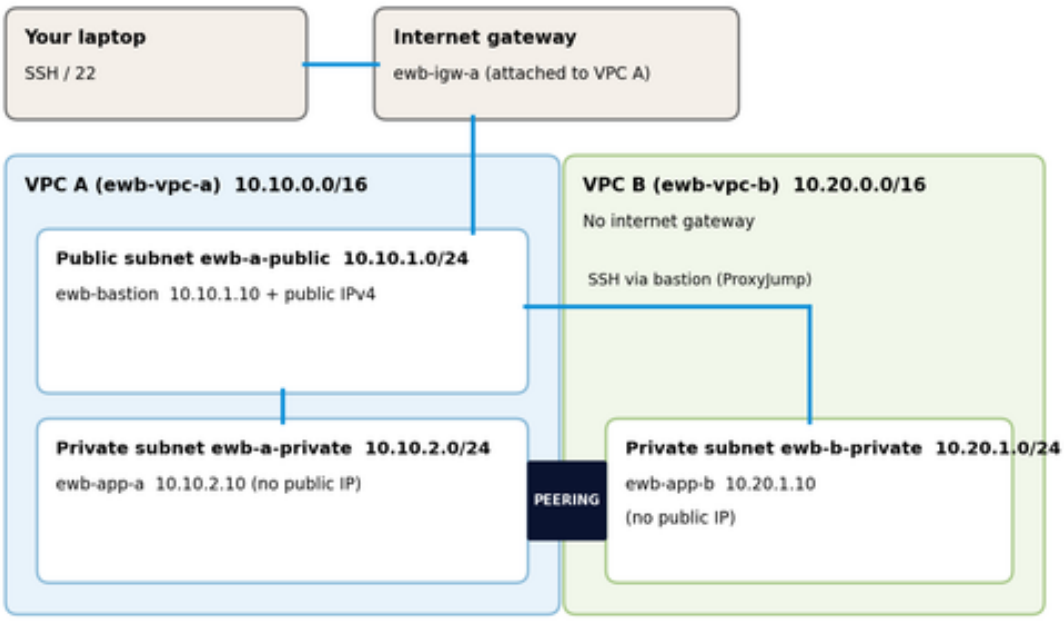
VPC Peering Project Report

Connect two VPCs. Prove private connectivity.

Student	Mayank Kumar Dubey
Batch	AWS-Internship Batch-1
Course	EWB Courses - AWS Architect Associate (Project 01)
Project tag	Project = EWB-VPC-Peering, Student = Mayank Kumar Dubey
AWS Region	Asia Pacific (Sydney) / ap-southeast-2 (Availability Zones ap-southeast-2b and ap-southeast-2c)
Completion date	24 October 2026

1. Architecture and inventory

Two VPCs with non-overlapping CIDRs (10.10.0.0/16 and 10.20.0.0/16) are joined by one VPC peering connection. Only the bastion in VPC A has a public IPv4 address. App-A and App-B stay private and talk to each other over private IPs through the peering connection.



Architecture: laptop reaches the bastion through the internet gateway, then App-A and App-B through ProxyJump. App-A and App-B talk over the peering connection.

Resource inventory (actual IDs from my account)

Resource	Name / ID	CIDR / IP
VPC A	ewb-vpc-a / vpc-04c426e9402df56ee	10.10.0.0/16
VPC B	ewb-vpc-b / vpc-05509ee7e07d38010	10.20.0.0/16
Subnets	ewb-a-public ([subnet ID]) ewb-a-private (subnet-029ac082218dcf93c) ewb-b-private (subnet-03dd5a862514a5280)	10.10.1.0/24 10.10.2.0/24 10.20.1.0/24
Internet gateway	ewb-igw-a ([igw ID]), attached to ewb-vpc-a	-
Route tables	ewb-a-public-rt (rtb-08aa92622d9b6b5df) ewb-a-private-rt (rtb-0904e0b8fd358c40f) ewb-rt-b-private (rtb-08478859ea39dcc13)	-
Peering connection	ewb-a-b-peer / pcx-0a3cecef3a458c5d1	Requester VPC A, accepter VPC B
ewb-bastion	i-055b7037557584c9b	10.10.1.10 + public IPv4 13.211.202.254
ewb-app-a	i-01c6a0dd09f7e21d9	10.10.2.10 (no public IP)
ewb-app-b	i-00b4495607dc5f4f7	10.20.1.10 (no public IP)
Security groups	ewb-sg-bastion (sg-0564cff2ef239ea50) ewb-sg-app-a (sg-030d4a7f0f67c6e12) ewb-sg-app-b (sg-06698482c823b4f33)	-
Key pair	ewb-lab (kept only on my laptop)	-

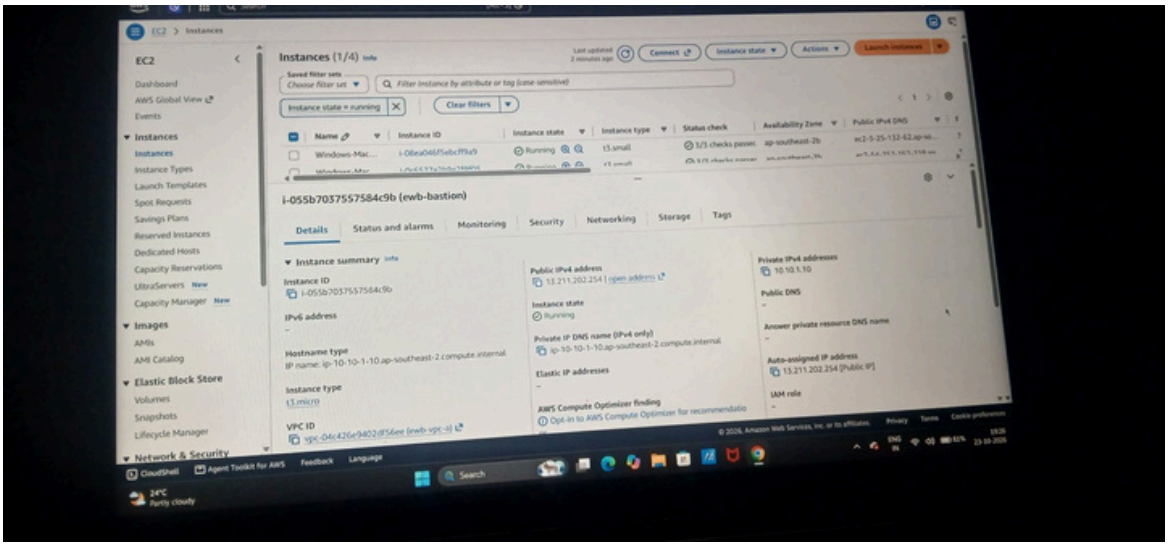


Figure 1 - Host: ewb-bastion. Instance running, public IPv4 13.211.202.254, private IPv4 10.10.1.10, in ewb-vpc-a.

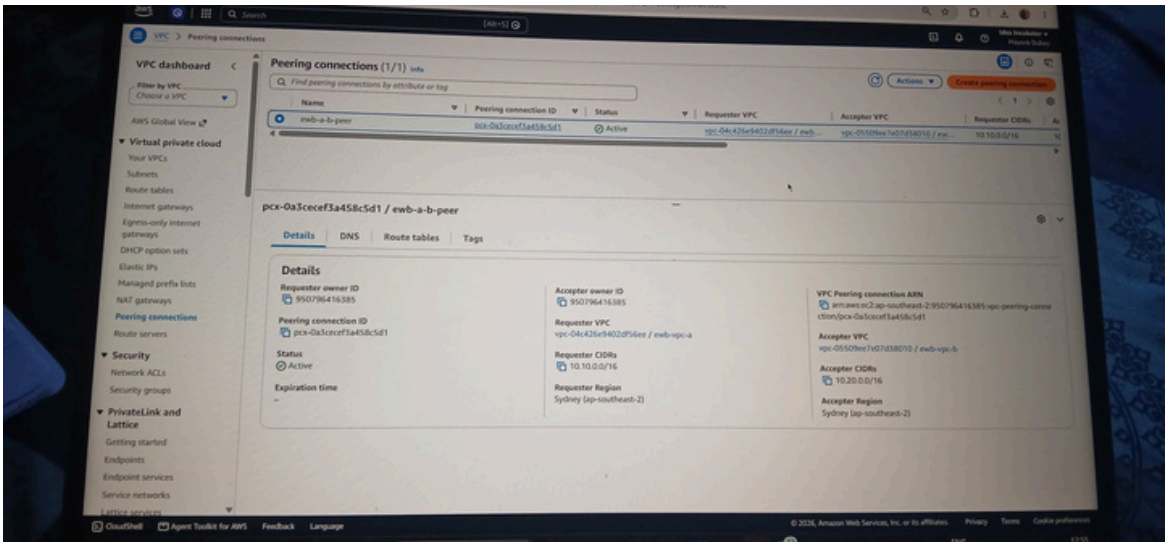


Figure 1b - Checkpoint 03. Host: AWS console. Peering connection ewb-a-b-peer (pcx-0a3cecef3a458c5d1) is Active. Requester: ewb-vpc-a (10.10.0.0/16). Accepter: ewb-vpc-b (10.20.0.0/16). Same account and Region (ap-southeast-2).

2. Subnets, route tables and security groups

Route tables (final configuration)

Route table / associated subnet	Destination	Target	Purpose
ewb-a-public-rt / ewb-a-public	10.10.0.0/16 0.0.0.0/0 10.20.0.0/16	local ewb-igw-a pcx-0a3cecef3a458c5d1	Bastion to App-A Bastion internet access Bastion SSH to App-B
ewb-a-private-rt / ewb-a-private	10.10.0.0/16 10.20.0.0/16	local pcx-0a3cecef3a458c5d1	Existing VPC A route App-A to VPC B
ewb-rt-b-private / ewb-b-private	10.20.0.0/16 10.10.0.0/16	local pcx-0a3cecef3a458c5d1	Existing VPC B route Return path to VPC A

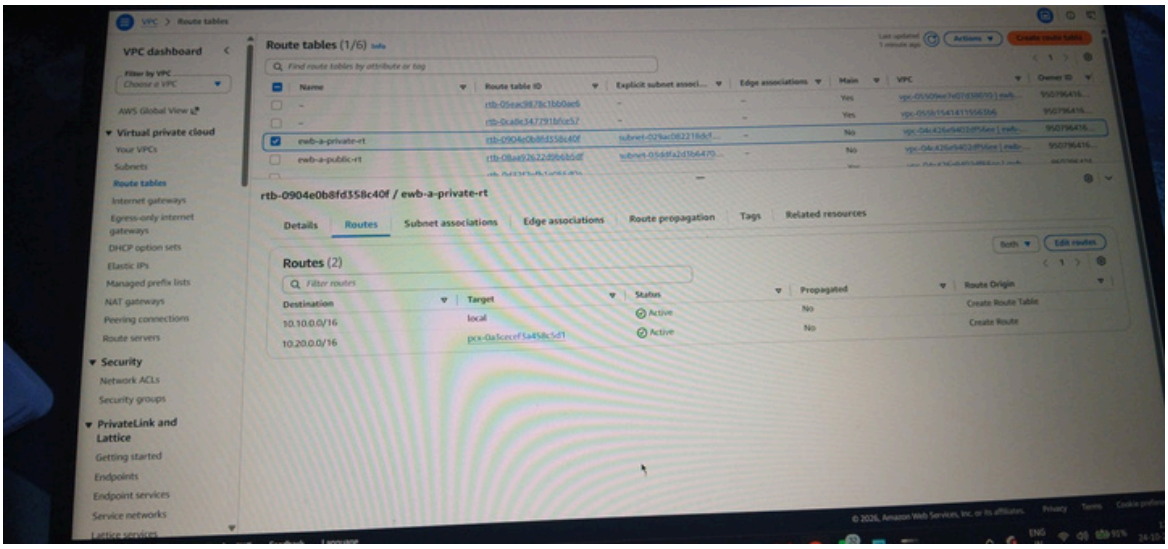


Figure 2 - Route table ewb-a-private-rt (rtb-0904e0b8fd358c40f), VPC ewb-vpc-a, explicit association with ewb-a-private. Routes: 10.10.0.0/16 local and 10.20.0.0/16 -> pcx-0a3cecef3a458c5d1, both Active.

Checkpoint 04 evidence: route tables and subnet association

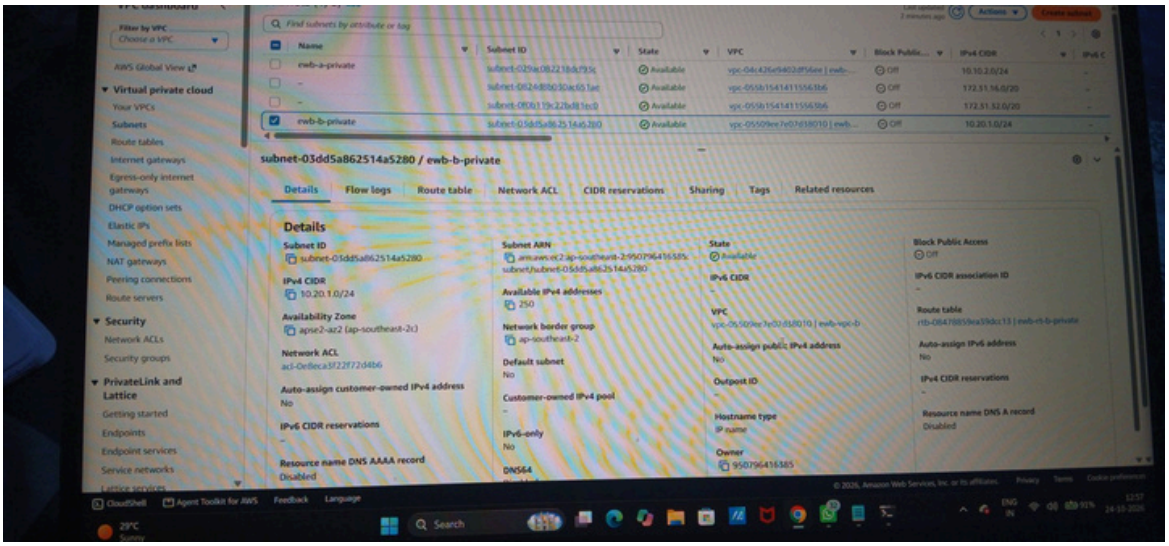


Figure 2e - Checkpoint 04. Host: AWS console. Subnet ewb-b-private (subnet-03dd5a862514a5280, 10.20.1.0/24) in ewb-vpc-b uses route table ewb-rt-b-private (rtb-08478859ea39dcc13). Auto-assign public IPv4 is No.

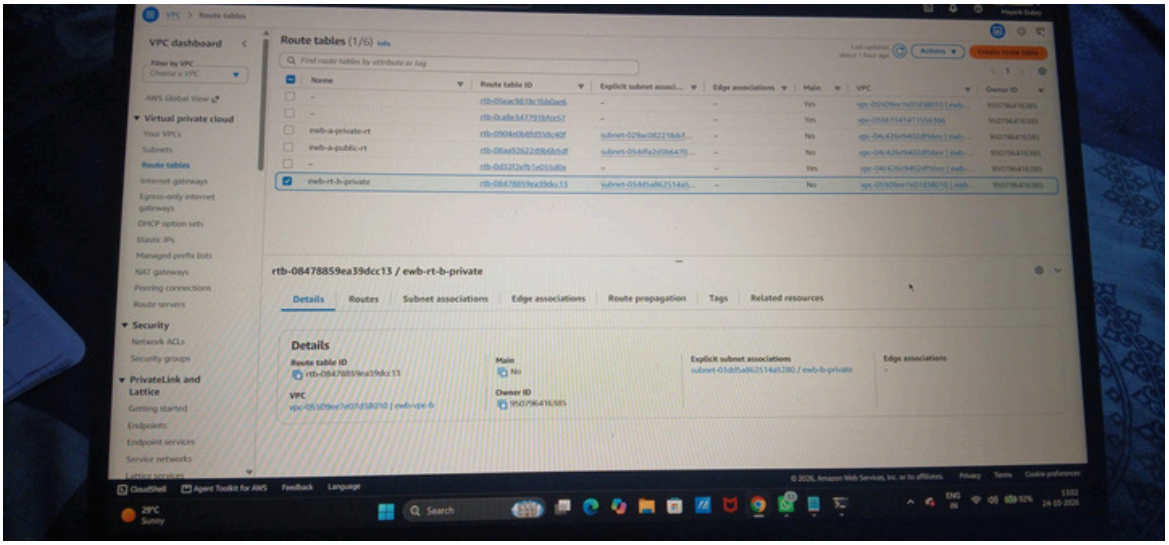


Figure 2f - Route table ewb-rt-b-private (rtb-08478859ea39dcc13) in ewb-vpc-b with explicit association to subnet ewb-b-private.

Security group inbound rules

Security group (VPC)	Type / port	Source
ewb-sg-bastion (VPC A)	SSH / TCP 22	[your public IPv4 /32]
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a (VPC A)	Custom ICMP IPv4 - Echo Request	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b (VPC B)	Custom ICMP IPv4 - Echo Request	10.10.2.10/32 (App-A)

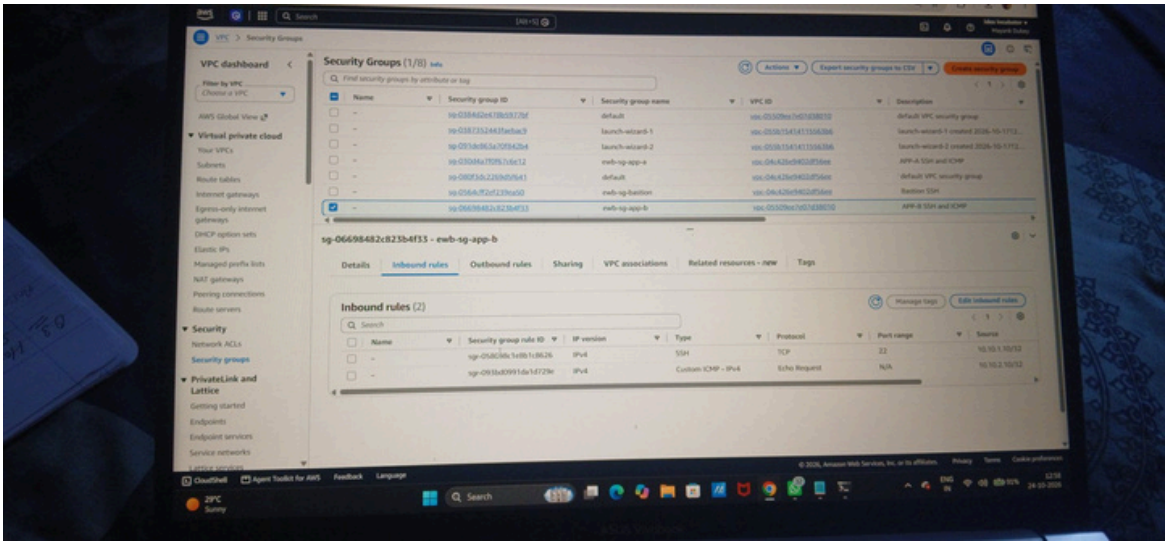


Figure 3c - Host: AWS console. ewb-sg-app-b (sg-06698482c823b4f33), VPC B. Inbound rules (2): Custom ICMP IPv4 Echo Request from 10.10.2.10/32 (App-A) and SSH / TCP 22 from 10.10.1.10/32 (bastion private IP).

3. Private SSH access (ProxyJump through the bastion)

The private key stays on my laptop (permissions restricted with icacls). The ~/.ssh/config file uses ProxyJump ewb-bastion, so App-A and App-B are reached through the bastion without copying any key onto an EC2 instance.

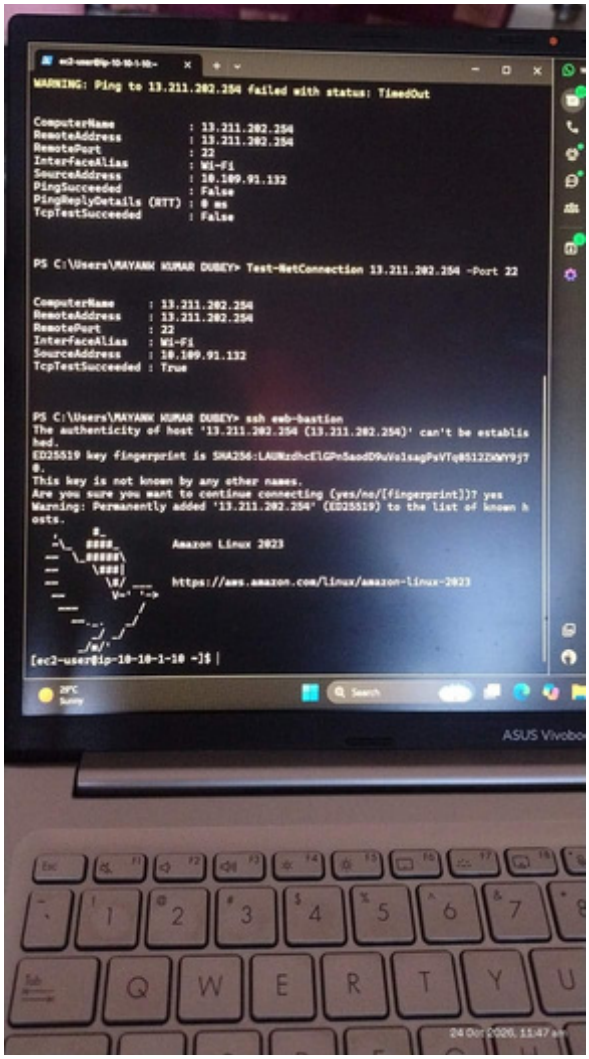


Figure 4 - Host: laptop -> ewb-bastion. ssh ewb-bastion succeeded; prompt ec2-user@ip-10-10-1-10.



Figure 6 - Test 04 Private SSH: ssh ewb-app-b reached App-B (10.20.1.10) through the bastion, prompt ec2-user@ip-10-20-1-10. This works only because the public route table has a route to 10.20.0.0/16 via the peering connection and VPC B has the return route.

4. Connectivity tests

Test	Where / command	Result
01 Baseline (before peering)	ewb-app-a: ping -c 4 10.20.1.10	[100% packet loss]
02 Forward path	ewb-app-a: ping -c 4 10.20.1.10	4 received, 0% loss
03 Reverse path	ewb-app-b: ping -c 4 10.10.2.10	4 received, 0% loss
04 Private SSH	laptop: ssh ewb-app-b	Shell on App-B through bastion
05 Isolation	EC2 instance details of App-A and App-B	No public IPv4 on either host
06 Controlled failure	Remove ICMP Echo rule on ewb-sg-app-b, ping from App-A, then restore	[100% loss, then 0% loss]

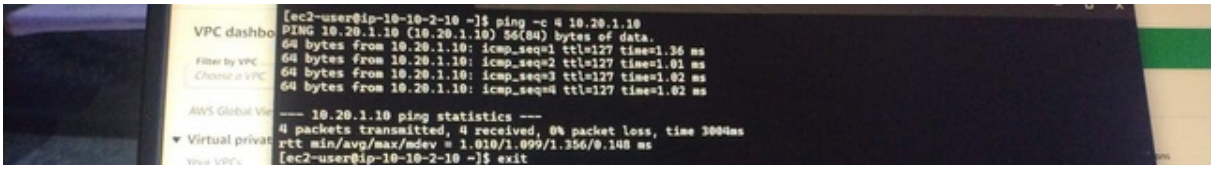


Figure 7 - Test 02 Forward path. Host: ewb-app-a (prompt ec2-user@ip-10-10-2-10). ping -c 4 10.20.1.10: 4 transmitted, 4 received, 0% packet loss.

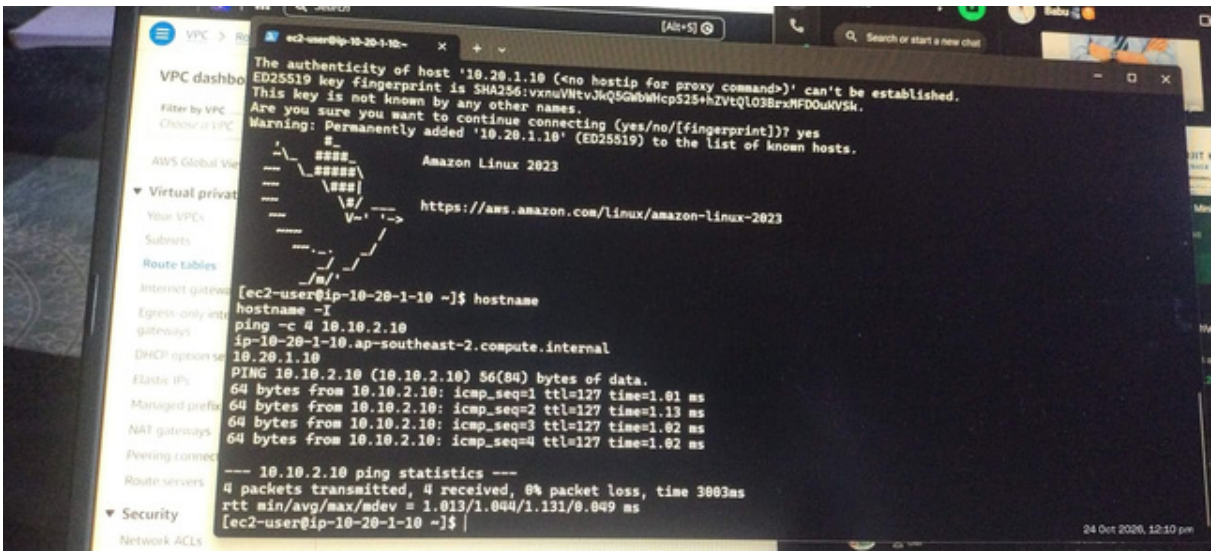


Figure 8 - Test 03 Reverse path. Host: ewb-app-b (hostname -I = 10.20.1.10). ping -c 4 10.10.2.10: 4 transmitted, 4 received, 0% packet loss.

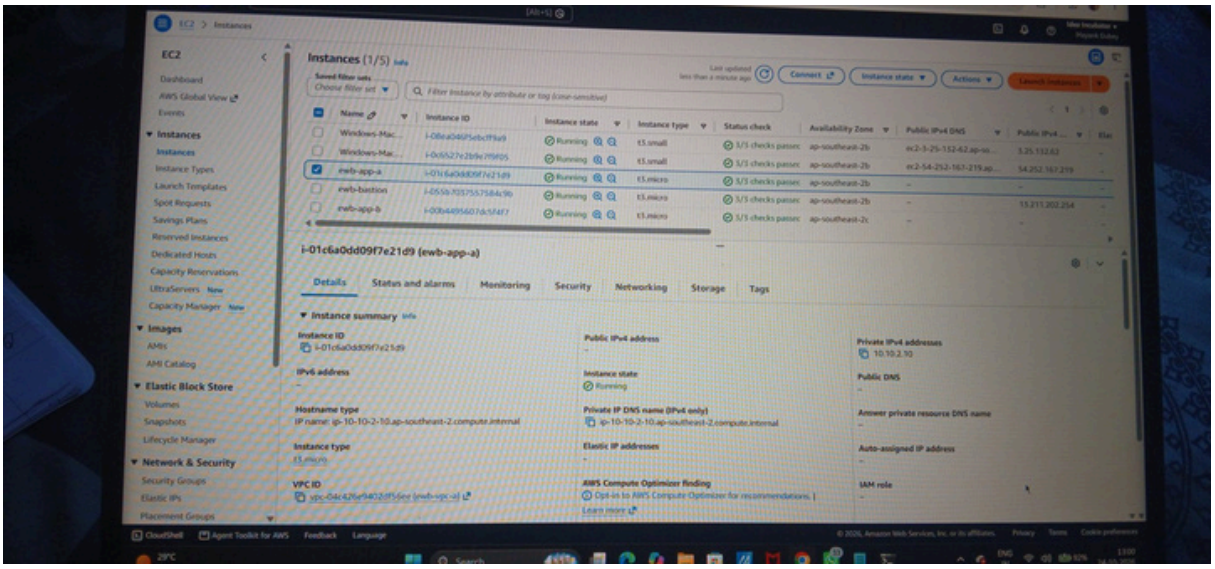


Figure 9 - Test 05 Isolation. ewb-app-a: Public IPv4 address empty, private IPv4 10.10.2.10, no Elastic IP. The list shows ewb-app-a, ewb-bastion and ewb-app-b Running.

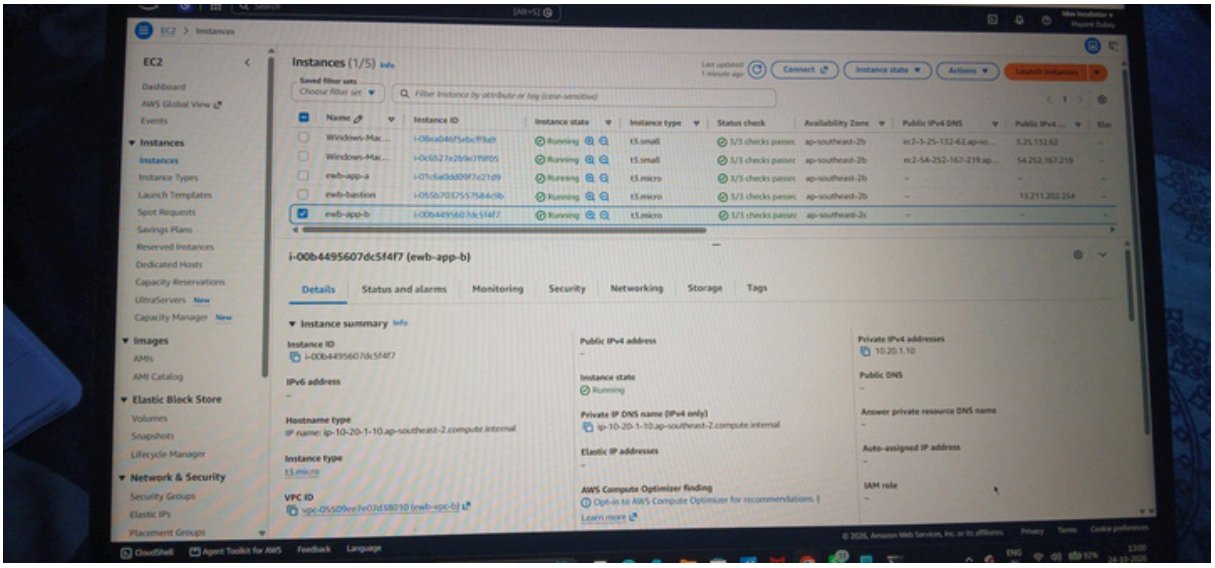


Figure 10 - Test 05 Isolation. ewb-app-b: Public IPv4 address empty, private IPv4 10.20.1.10, no Elastic IP.

Controlled failure (ICMP security group rule)

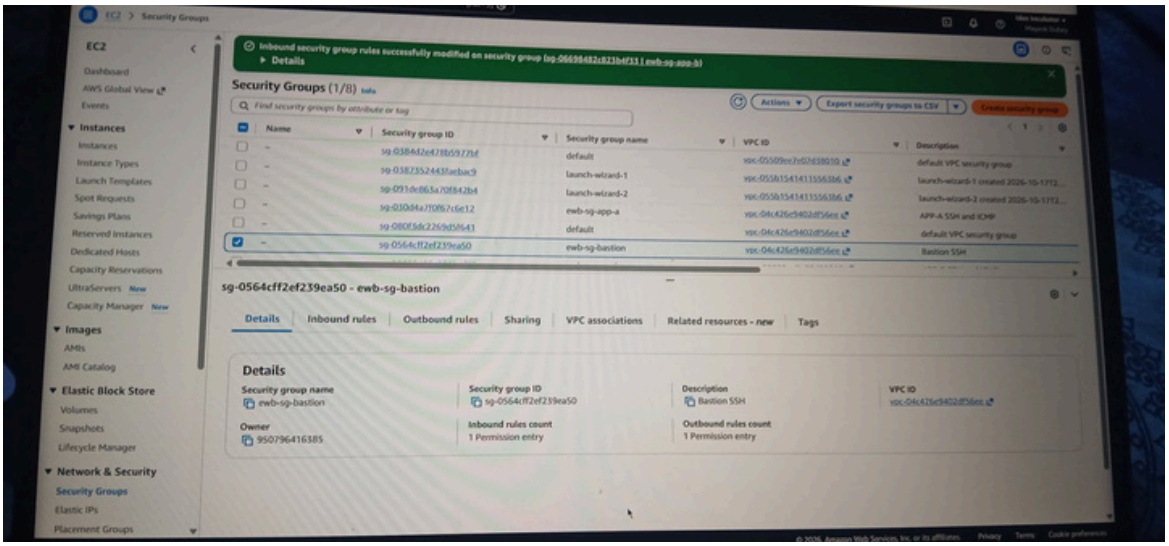


Figure 12b - Host: AWS console. Inbound security group rules successfully modified on ewb-sg-app-b (sg-06698482c823b4f33).

5. Forward path, return path and what I learned

Forward path

A ping from App-A (10.10.2.10) to App-B (10.20.1.10) leaves the private subnet. ewb-a-private-rt matches 10.20.0.0/16 and sends it to the peering connection pcx-0a3cecef3a458c5d1. In VPC B the packet reaches ewb-b-private, and ewb-sg-app-b allows ICMP Echo Request from 10.10.2.10/32.

Return path

The reply goes from 10.20.1.10 to 10.10.2.10. ewb-rt-b-private matches 10.10.0.0/16 and sends it back over the same peering connection. Security groups are stateful, so the reply is allowed automatically. Without this return route the ping would still fail, because a one-way route is not enough.

Why the SSH source on App-B is the bastion private IP

With ProxyJump the bastion opens a second SSH connection to App-B, so App-B sees 10.10.1.10 as the source. The public route table also needs 10.20.0.0/16 -> pcx so that the bastion can reach VPC B.

Other answers

Active peering does not guarantee a ping: route tables and security groups must also allow it. VPC B cannot use VPC A's internet gateway over peering, because peering is non-transitive and does not provide internet access.

Troubleshooting note: bastion SSH timeout

At first the connection test to the bastion on port 22 timed out (TcpTestSucceeded False). After I updated the source rule in ewb-sg-bastion, Test-NetConnection returned TcpTestSucceeded True and ssh ewb-bastion worked (Figure 16).

Add one or two lines in your own words: which address you allowed in the rule and why it differed from the first one you tried.

Keep the final rule as your own address with /32, never 0.0.0.0/0.

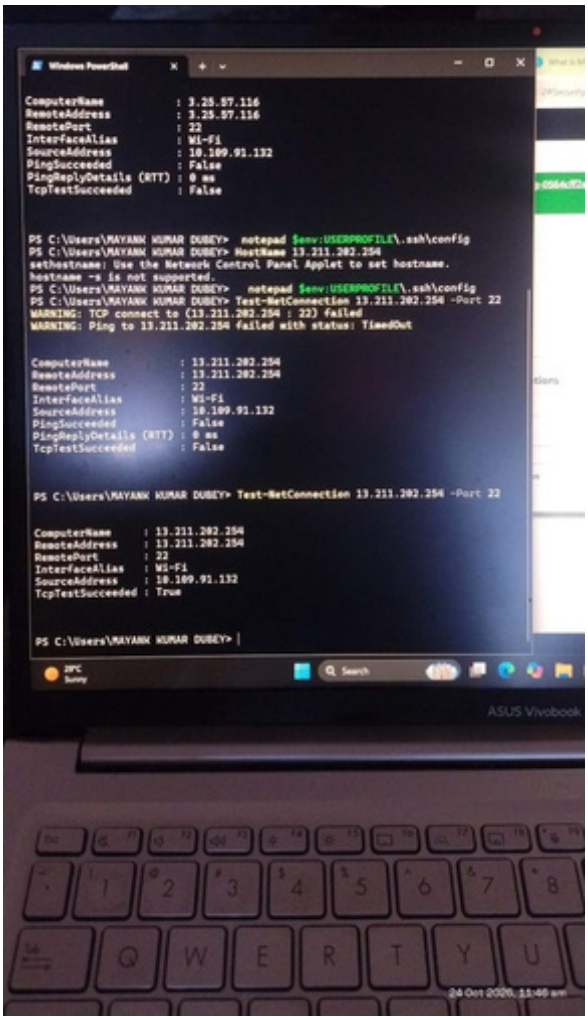


Figure 16 - Host: laptop. Test-NetConnection to the bastion on port 22 failed first (TcpTestSucceeded False) and then succeeded (True) after the security group fix, followed by ssh ewb-bastion.

What I learned

1) A VPC peering connection needs routes on both sides. 2) Security groups on the destination must allow the peer's private IP. 3) [add your own point, for example about the bastion security group source address]. 4) Testing a failure first, and then fixing it, proves which component makes the connection work.